

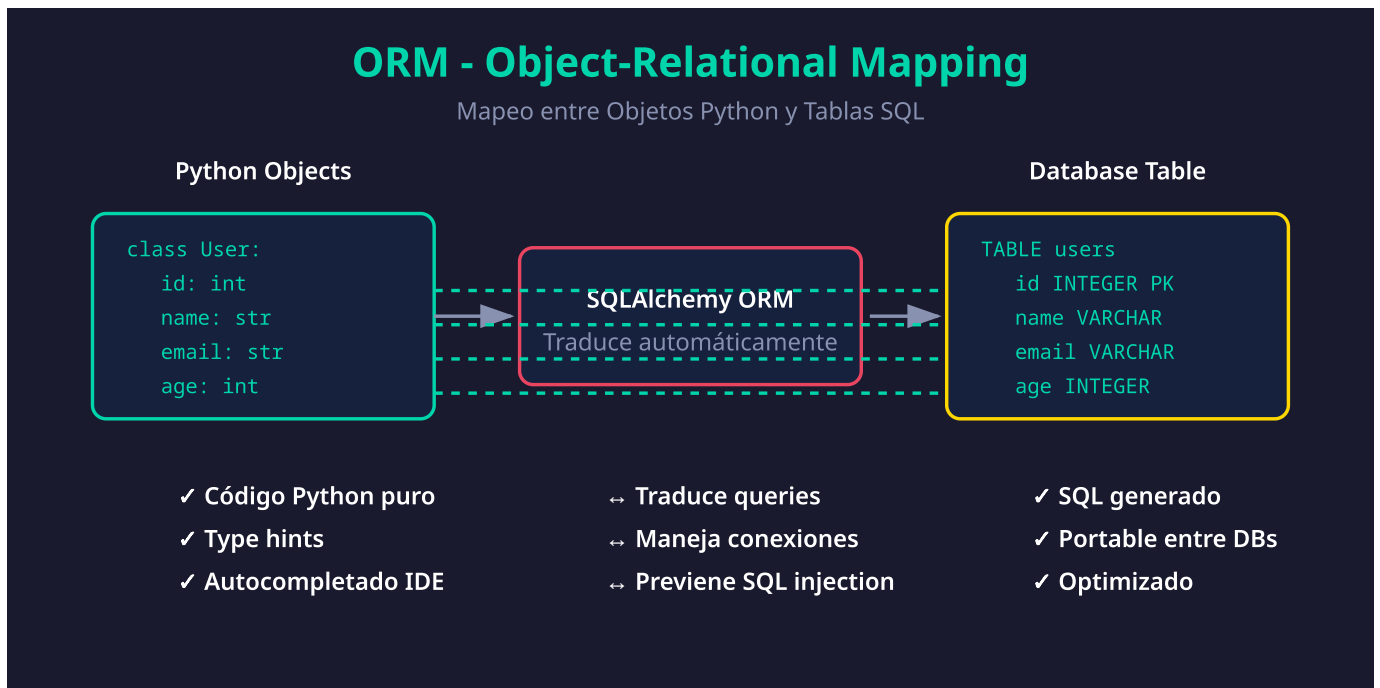
Introducción a ORM

Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

- ✓ Entender qué es un ORM y por qué usarlo
- ✓ Conocer las ventajas y desventajas de los ORMs
- ✓ Entender por qué elegimos SQLAlchemy
- ✓ Diferenciar entre SQLAlchemy Core y ORM

Contenido



1. ¿Qué es un ORM?

ORM (Object-Relational Mapping) es una técnica que permite interactuar con bases de datos relacionales usando objetos de programación en lugar de SQL directo.

```
# ❌ Sin ORM - SQL directo (propenso a errores, SQL injection)
cursor.execute("""
    INSERT INTO users (name, email, age)
    VALUES (?, ?, ?)
""", ("John", "john@example.com", 25))

# ✅ Con ORM - Objetos Python (seguro, tipado, mantenible)
user = User(name="John", email="john@example.com", age=25)
session.add(user)
session.commit()
```

2. El Mapeo Objeto-Relacional

Concepto SQL

Concepto ORM

Tabla	Clase (Model)
Columna	Atributo
Fila	Instancia (objeto)
Foreign Key	Relación
Query	Método/Expresión

```
# La tabla "users" se convierte en la clase User
class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True)
    age: Mapped[int | None] = mapped_column(default=None)

# Una fila se convierte en un objeto
user = User(name="John", email="john@example.com")
print(user.name) # "John" - acceso como atributo
```

3. Ventajas de Usar un ORM

✓ Seguridad

```
# ✗ SQL Injection vulnerable
query = f"SELECT * FROM users WHERE email = '{user_input}'"
# Si user_input = "'; DROP TABLE users; --" → Desastre!

# ✓ ORM previene SQL injection automáticamente
stmt = select(User).where(User.email == user_input)
# Los parámetros siempre se escapan correctamente
```

✓ Productividad

```
# Código más corto y legible
user = User(name="John", email="john@example.com")
session.add(user)
session.commit()

# vs escribir INSERT statements manualmente
```

✓ Portabilidad

```
# El mismo código funciona con diferentes bases de datos
# Solo cambia la URL de conexión

# SQLite (desarrollo)
SQLALCHEMY_DATABASE_URL = "sqlite:///./app.db"
```

```
# PostgreSQL (producción)
SQLALCHEMY_DATABASE_URL = "postgresql://user:pass@localhost/db"
```

✓ Mantenibilidad

```
# Modelos centralizados, fáciles de modificar
class User(Base):
    __tablename__ = "users"

    # Agregar un campo es simple
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
```

4. Desventajas de los ORMs

⚠ Curva de Aprendizaje

```
# Necesitas aprender la API del ORM
# SQLAlchemy tiene muchos conceptos:
# - Engine, Session, Connection
# - Mapped, mapped_column
# - select, insert, update, delete
# - Relationships, lazy loading, eager loading
```

⚠ Abstracción con Costo

```
# Queries complejas pueden ser menos eficientes
# El ORM genera SQL que quizás no es óptimo

# A veces necesitas SQL raw para optimización
stmt = text("SELECT * FROM users WHERE ...")
result = session.execute(stmt)
```

⚠ Debugging Más Difícil

```
# Errores pueden venir del ORM, no de tu código
# Necesitas entender qué SQL se genera

# Tip: Habilitar logging de SQL
import logging
logging.getLogger('sqlalchemy.engine').setLevel(logging.INFO)
```

5. ¿Por Qué SQLAlchemy?

SQLAlchemy es el ORM más maduro y poderoso de Python:

Característica	SQLAlchemy	Otros ORMs
Madurez	18+ años	Variable
Async nativo	✓ (2.0+)	Parcial
Type hints	✓ Completo	Parcial

Flexibilidad	Alta (Core + ORM)	Limitada
Comunidad	Muy grande	Variable
Documentación	Excelente	Variable

```
# SQLAlchemy 2.0 - Estilo moderno con type hints
from sqlalchemy.orm import Mapped, mapped_column

class User(Base):
    __tablename__ = "users"

    # Type hints integrados - IDE sabe el tipo
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
```

6. SQLAlchemy Core vs ORM

SQLAlchemy tiene dos capas:

Core (Bajo nivel)

```
from sqlalchemy import Table, Column, Integer, String, MetaData, insert

metadata = MetaData()

users = Table(
    "users",
    metadata,
    Column("id", Integer, primary_key=True),
    Column("name", String(100)),
)

# Operaciones con tablas directamente
stmt = insert(users).values(name="John")
connection.execute(stmt)
```

ORM (Alto nivel) - Lo que usaremos

```
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))

# Operaciones con objetos
user = User(name="John")
session.add(user)
```

💡 **En este bootcamp usamos el ORM** porque es más intuitivo y productivo para aplicaciones web.

7. Sync vs Async

SQLAlchemy 2.0 soporta ambos modos:

Síncrono (más simple)

```
from sqlalchemy import create_engine
from sqlalchemy.orm import Session

engine = create_engine("sqlite:///./app.db")

with Session(engine) as session:
    user = session.get(User, 1)
```

Asíncrono (mejor rendimiento)

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession

engine = create_async_engine("sqlite+aiosqlite:///./app.db")

async with AsyncSession(engine) as session:
    result = await session.get(User, 1)
```

💡 **Comenzaremos con sync** para entender los conceptos. Luego migraremos a async.

Verificación de Conceptos

Responde mentalmente:

1. ¿Qué problema resuelve un ORM?
 2. ¿Cómo previene SQL injection un ORM?
 3. ¿Cuál es la diferencia entre Core y ORM en SQLAlchemy?
 4. ¿Por qué SQLAlchemy es popular en el ecosistema Python?
-

Recursos Adicionales

- [SQLAlchemy ORM Tutorial](#)
 - [What's New in SQLAlchemy 2.0](#)
 - [FastAPI SQL Databases](#)
-

Checklist

- ☐ Entiendo qué es un ORM
 - ☐ Conozco las ventajas de usar SQLAlchemy
 - ☐ Sé la diferencia entre Core y ORM
 - ☐ Entiendo por qué comenzamos con sync
-

[Siguiente: Configuración de SQLAlchemy →](#)